

Code Instruction Selection Based on SSA-Graphs

Erik Eckstein¹, Oliver König¹, and Bernhard Scholz²

¹ ATAIR Software GmbH, Vienna, Austria
{eckstein,koenig}@atair.co.at

² Institute of Computer Languages, Vienna University of Technology, Austria
scholz@complang.tuwien.ac.at

Abstract. Instruction selection for embedded processors is a challenging problem. Embedded system architectures feature highly irregular instruction sets and complex data paths. Traditional code generation techniques have difficulties to fully utilize the features of such architectures and typically result in inefficient code.

In this paper we describe an instruction selection technique that uses static single assignment graphs (SSA-graphs) as underlying data structure for selection. Patterns defined as graph grammar guide the instruction selection to find (nearly) optimal results. We present an approach which maps the pattern matching problem to a *partitioned boolean quadratic optimization problem* (PBQP). A linear PBQP solver computes optimal solutions for almost all nodes of a SSA-graph.

We have implemented our approach in a production DSP compiler. Our experiments show that our approach achieves significant better results compared to classical tree matching.

1 Introduction

Highly specialized processors such as digital signal processors (DSP) or micro controller systems feature irregularities in their instruction sets. Therefore code generation for these processors is still a research topic and is not satisfying solved so far.

In a traditional compiler framework code generation is decomposed in several sub-problems. The main building blocks of a code generator are *instruction selection*, *instruction scheduling*, and *register allocation*. First, a front end of a compiler translates the source program into an *intermediate representation*. After performing high-level optimizations, the instruction selector translates the intermediate representation into target code. Instruction scheduling reorders the target code to keep register pressure low and to utilize pipelining and parallel units of the target architecture. Register allocation assigns hardware registers to pseudo registers. Beside these three building blocks, most compilers for embedded systems also perform additional optimizations to utilize target dependent hardware features, e.g. addressing modes [3].

Tree pattern matching is a widely used technique for instruction selection [1]. Usually the unit of translation is a statement which is represented as a data flow tree (DFT). A set of rules is used to match the DFT. The matcher selects those rules such that the sum of all applied rule costs is a minimum. An algorithm for tree pattern matching has two phases: labeling and reducing. In the labeling phase minimal costs are calculated for each node and each non-terminal. This is done by checking each non-terminal combination in a bottom-up walk of the tree. In the reduction phase the tree is traversed top-down and the rules with minimal costs are selected. The tree matching algorithm employs *dynamic programming* firstly introduced by BEG [8] and BURG [6]. The dynamic programming approach is performed in linear time. Though this technique is fast, it does not consider the computational flow of a function.

DAG matching is an extension to tree matching. Instead of trees, directed acyclic graphs are considered. DAG matching is a NP-complete problem. A proof for NP completeness of matching DAGs is given by [11]. In the work of Ertl [4] an approach is presented, which modifies the tree pattern matcher algorithm so that it can be used on DAGs. A checker proves whether the DAG matching algorithm yields optimal results for a specific grammar. This approach differs from our approach in some points: First, the algorithm does code duplication. Second, it is not possible to perform the algorithm on a graph containing cycles, because it still relies on the bottom-up and top-down phases of the tree pattern matcher. DAG matching was also mapped to the binate covering problem [10]. However, DAG matching still does not consider the computational flow of functions.

Beside the dynamic programming method, there are a number of specialized approaches for code generation with pattern matching. Leupers introduced code selection for SIMD instruction generation, based on integer linear programming [9].

This paper presents a new technique for instruction selection of code generators. In contrast to previous approaches the computational flow of a whole function is taken into account. For representing the computational flow the SSA-graph is used which combines data flow trees (DFT) and def-use relations of a function. An ambiguous grammar describes possible derivations of the SSA-graphs. Production rules have cost terms and code templates. Cost terms are used to find the derivation with minimal overall costs. Unlike conventional approaches, parsing SSA-graphs is more difficult since cycles are allowed in the graphs.

Parsing generic graphs is NP-complete since even parsing DAGs is NP-complete [11]. To get a handle on the problem, we map the instruction selection problem for SSA-graphs to *partitioned boolean quadratic problem* (PBQP). The basic concept of our SSA-graph matching algorithm is shown in Figure 1. First, the SSA-graph with its ambiguous grammar is mapped to PBQP. Second, the PBQP solver computes the grammar derivation with minimal costs. Third, based on the grammar derivation code is produced.

Note that the PBQP solver consists of two phases: In the first phase the graph is reduced until a trivial solution remains. In the second phase the solution is